

Project Gap Prevention — User-Facing Reliability Audit

Full-project scan of `app/` (player PWA), `api/` (Hono + MongoDB), and `web/` (owner dashboard + booking). Goal: eliminate the operational gaps that would let an **ordinary user** hit a visible bug — white screens, permanent spinners, "undefined"/"NaN" values, dead-ends, silent failures, and a broken booking/money path. Every finding below was confirmed by reading the cited code, not inferred.

Scan date: 2026-07-21

Executive summary

The codebase is, on the whole, **unusually defensive** — most screens have explicit loading/error/empty states, list wrappers coalesce to `?? []`, money/date formatters guard against `NaN / Invalid Date`, and external calls (email, push, geocoding) swallow their own failures. The API has a solid global `onError` handler that prevents stack-trace leaks and process crashes.

The real risk clusters into a handful of **systemic patterns** rather than scattered one-offs. Fixing the six cross-cutting themes below neutralizes the large majority of user-visible failure modes.

The two things that will bite a real user today (fix first)

- `web/` **booking** → **checkout path is broken end-to-end**. The public "Book a court" flow reads dummy JSON (dead-ends on "Venue not found" for real venues), and checkout **always** shows "Payment confirmed" while **never creating a booking**. This is the money path and it does not work against real data.
- No React error boundary exists in either `app/` or `web/`**. Any single render throw white-screens the entire app with no recovery. This is what turns every latent null-access bug in ~130 screens into a fatal crash instead of a contained error card — the single highest-leverage fix.

Cross-cutting themes (highest leverage — fix the pattern, not just the instance)

#	Theme	Where	Fix once, applies everywhere
A	No error boundaries	<code>app/</code> (main.tsx/App.tsx), <code>web/</code> (main.jsx/router.jsx)	Add a top-level <code>ErrorBoundary</code> (+ per-screen/route <code>errorElement</code>) with a "Something went wrong — Reload" fallback. Special-case lazy chunk-load failures to force reload.
B	Fail-silent-as-empty	owner dashboard hooks, feed, clubs directory	A failed fetch renders the *empty* state ("No bookings yet", "No posts yet", "No clubs found") indistinguishable from genuinely empty, with no retry. Track a per-list error flag; render an <code>ErrorState</code> w/ retry when the request rejected.
C	Optimistic UI without rollback	feed hide/delete/not-interested	UI shows fake success (post hidden/deleted) even when the server call failed. Capture prior state, restore in <code>.catch</code> .
D	Check-then-write races	bookings, coach-bookings, waitlist, game-join, roster add	Non-atomic read-modify-save allows double-booking / over-capacity / lost updates. Use unique partial indexes (catch <code>E11000</code> → 409) or atomic <code>findOneAndUpdate</code> guards.
E	Bad user input → 500 instead of 400/404	~265 <code>findById</code> sites, ~111 <code>c.req.json()</code> sites, search regex	Invalid ObjectId → <code>CastError</code> 500; empty/malformed JSON body → <code>SyntaxError</code> 500; raw <code>new RegExp(userInput)</code> → 500 + ReDoS. Add shared <code>asObjectId()</code> , <code>readJson()</code> helpers and escape regex input.
F	Silent mutation failures	organizer/roster manage actions	Approve/check-in/cancel/remove use <code>try/finally</code> (no catch) — on failure the action silently does nothing, no toast. Add <code>catch</code> + error surface.

`web/` — Owner dashboard + public booking (React + react-router)

Critical

W1. Public booking flow dead-ends on "Venue not found" for every real venue `web/src/features/bookings/BookingPage.jsx:17` (+ `shared/data/index.js:17`) The "Book a court" CTA (`VenueDetailPage.jsx:224`) links to `/venues/:slug/book`, but `BookingPage` resolves the venue from **dummy JSON** (`dummies/venues.json`) while the detail page uses the **real API** (`fetchVenueBySlug`). Any real venue whose slug isn't in the dummy file shows `🙄 Venue not found`; on a slug collision the courts/prices are fake. **Fix:** back `BookingPage` with the real `fetchVenueBySlug` + `fetchCourts` and drop the `shared/data` dummy dependency.

W2. Checkout always shows "Payment confirmed" but never creates a booking `web/src/features/bookings/CheckoutPage.jsx:54-58, 197-217` `handlePay` does `setTimeout(... setStep(2))` — it never calls the API in test *or* live mode. The user always reaches "Payment confirmed! View bookings", but nothing is persisted; "View bookings" then lands on an empty list. There is no failure branch at all. **Fix:** wire the documented seam (`apiPost('/api/v1/payments/checkout', ...)`), advance to step 2 only on success, add an error state.

High

W3. No error boundaries anywhere — `main.jsx`, `App.jsx`, `router.jsx` No route declares `errorElement`; any render throw falls to react-router's bare "Unexpected Application Error". A post-redeploy lazy-chunk 404 hits the same generic page instead of a reload prompt. **Fix:** top-level `errorElement` (+ per-layout) with a branded retry/reload; special-case chunk-load errors to force reload.

W4. Owner overview/insights white-screen if any venue's analytics lacks `kpis` `web/src/features/owner/useOwnerData.js:121-131`, `OwnerInsightsPage.jsx:32-34, 44` The reducer/memos dereference `a.kpis.revenue.month`, `a.kpis.occupancyPct.week` with no optional chaining, running during render for every venue. One venue missing a full

`kpis` shape crashes all of `/owner` and `/owner/insights`. **Fix:** optional-chain with numeric defaults — `a?.kpis?.revenue?.month ?? 0`.

W5. RequirePermission re-fetches /auth/me and full-screen "Loading..." flashes on every in-console navigation

`web/src/features/auth/RequirePermission.jsx:13-27, 33-39` Effect deps include `location.pathname / search` and it `setChecked(false)` before each `refreshMe()`; since `/owner`, `/admin`, `/coach`, `/organizer` are each wrapped, every tab click blanks the whole layout and fires a redundant `/auth/me`. **Fix:** run `refreshMe()` once on mount (or throttle); don't reset `checked` on path change.

Medium

- W6. No 404/catch-all inside authenticated consoles → blank content pane** — `router.jsx:198-294`. Mistyped `/owner/bogus` renders the layout with an empty `<Outlet>`. Add `{ path: '*', element: <NotFoundPage/> }` to each console block.
- W7. Chat send failures silently swallowed** — `ChatPage.jsx:50-55` (`catch { /* ignore */ }`). Message vanishes, no retry. Set an error state + "tap to retry".
- W8. format.js money() throws on explicit null currency** — `format.js:9-10`. Default only covers `undefined`; `null` → `Cannot read properties of null`. Coerce `String(currency || 'PHP')`.
- W9. BookingPage price shows \$undefined / passes undefined amount** — `BookingPage.jsx:42, 92, 105`. Court missing `pricePerHour` → `$undefined/hr` and a \$0 "payment". Default to `0` /hide and validate amount before checkout.
- W10. Optimistic conversation delete can resurrect a stale row** — `MessagesPage.jsx:43-48`. Concurrent deletes race on captured `prev`. Key rollback off a functional update or refetch on error.
- W11. Currency symbol mismatch (\$ vs ₱)** — `BookingRefundPage.jsx:6-9` vs `CheckoutPage.jsx:20`. Same booking shows `$15` at checkout, `₱15` at refund. Single shared money formatter + one currency source.
- W12. Demand heatmap emits NaN alpha on zero demand** — `DemandTab.jsx:42-43`. `v/max` with `max===0`. Degrades to transparent (no crash) but guard `max || 1`.

Checked & clean: no localhost/dev URL leak (`client.js:5` → prod host); `useClubStream.js` has cleanup + capped backoff; `BookingsInbox / ReviewsInbox / MyBookingsPage / Messages / Chat / LeakageTab / DemandTab` have loading/error/empty + abort guards; forms use `StatusButton` (disabled while saving → no double-submit).

DONE — all web findings resolved (2026-07-21)

Plain-language summary of what was fixed in the owner dashboard + public booking site:

#	Level	What was wrong (plain language)	What we did
L1	—	The "Sign In" button opened the website's own login.	It now sends users to the mobile-app login (PWA), where accounts live.
W1	Critical	The "Book a Court" page used fake demo data — every real venue showed "Venue not found."	It now loads the real venue and its courts from the live system, with proper loading / error / "no courts yet" states.
W2	Critical	Checkout always said "Payment confirmed" but never actually booked anything — money path was broken.	Checkout now really creates the booking and records the payment, only shows success when it works, and shows an error if it fails.
W3	High	Any single screen error blanked the whole site .	Added a safety net: a broken page shows a friendly error card (or a "reload for the new version" prompt), not a white screen.
W4	High	The owner overview/insights crashed to a blank screen if one venue's stats were incomplete.	Made the stats calculations safe so missing data just reads as 0 instead of crashing.
W5	High	Every click inside the owner/admin console flashed a full-screen "Loading..." and re-checked the login.	Now it checks once when you enter — no more flashing on every tab click.
W6	Medium	A mistyped console URL (e.g. <code>/owner/bogus</code>) showed a blank content pane .	Now shows a proper "page not found" inside every console.
W7	Medium	If a chat message failed to send, it disappeared silently with no retry.	Now shows an error with a Retry button (and keeps your typed text).
W8	Medium	The money formatter could crash on certain currency values.	Made it safe against empty/null currency.
W9	Medium	A court missing a price showed "\$undefined" and could pass a \$0 payment.	Missing prices now default cleanly and the amount is validated before checkout.
W10	Medium	Deleting a conversation could bring a deleted one back if two actions overlapped.	Fixed the undo logic so a failed delete only restores that one row.
W11	Medium	The same booking showed \$ at checkout but ₱ at refund — inconsistent currency.	All booking pages now use one shared money format (₱).
W12	Medium	*(reported: heatmap divide-by-zero)*	Already safe — the guard already exists. No change needed.

Build:  passes, 0 errors. 12 files changed.

Global error handling (context for severity): `app.onError` (`index.ts:167` , `shared/middleware/error-handler.ts`) turns `ZodError` → clean **400**, `HTTPException` → its status, and **everything else** (`CastError`, `TypeError`, `SyntaxError`, `E11000`) → generic **500** with **no stack leak in production**. So unguarded throws don't crash the process or leak stacks — they return the **wrong status** (usually should be 400/404). The concurrency queue releases slots in `finally` ; the scheduler wraps job bodies in `try/catch` and `unref` s timers. Unhandled rejections / SSE / external email / push were checked and are well-handled.

● **High**

- A1. Double-booking race — venue can oversell the last court** `api/src/features/bookings/bookings.controller.ts:649-654 → :778` (`createBooking`) `findSlotConflict` (read) then `Booking.create` (write) with no `txn/lock` — two near-simultaneous bookings for the last court both pass and both persist. (The code comment already concedes the window.) **Fix:** unique partial index on `{venueId,courtId,date,startTime}` (catch `E11000` → 409), or a transaction wrapping check + insert.
- A2. Same race on coach bookings** — `coach-bookings.controller.ts:98-105` . `findOne` clash check then `create` → coach double-booked. Unique partial index on `{coachId,date,startTime}` over blocking statuses.
- A3. Unvalidated `startTime` / `endTime` → NaN bypasses price validation** `bookings.controller.ts:320-321` (schema), `:682-684` , `:760` `startTime` is a bare `z.string().optional()` (coach & modify schemas have the `^d{2}:\d{2}$` regex). A malformed time makes `expectedTotal` NaN; the `Math.abs(clientAmount - expectedTotal) > 1` check is `NaN > 1 === false` , so **the booking is created at any client-chosen amount**. **Fix:** constrain with `^d{2}:\d{2}$` and `Number.isFinite` guard before the price comparison.
- A4. Search builds `new RegExp(userInput)` — invalid-regex 500 + ReDoS** `search.controller.ts:41` (also `:62` , `:80` , `:109` , `:139`) An unbalanced `( / [` throws → 500; a crafted input `( (a+)$ )` pins the event loop for all users. **Fix:** escape input `( q.replace(/[.*?^$()|[\]\|\\|/g, '\\$&') )` or use only the `$text` index path.
- A5. Coaches can self-publish an unverified profile (moderation bypass)** `coaches.controller.ts:54` , `:182` , `:289-290` `isListed` is settable via `PATCH /coaches/me` ; `listCoaches` filters only `{isListed:true}` and never checks `isVerified` . A never-verified coach can appear in the public directory instantly. **Fix:** gate `listCoaches` on `isVerified:true` , or strip `isListed` from the self-update schema.
- A6. Game join is non-atomic → over-capacity / double-join** `games.controller.ts:564-584` `if (participantIds.length >= seats)...; push; save()` — two players on the last seat both read pre-full and both save; a double-tap can list the same user twice. **Fix:** `Game.findOneAndUpdate({_id, $expr:{ $lt: [{ $size: '$participantIds' }, seats] }, participantIds: { $ne: user.sub }, { $addToSet: { participantIds: user.sub }, { new: true } })` → 409 on null.

● **Medium**

- A7. Roster add/remove lost updates** — `rosters.controller.ts:95-101` , `110-112` . Full-array read-modify- `save()` clobbers a concurrent add. Use atomic `$push` / `$pull` via `findOneAndUpdate` .
- A8. Waitlist claim non-atomic + books P0** — `waitlist.controller.ts:121-146` . Re-check then create over-capacity; creates a `confirmed` , `amount:0` booking holding a court unpaid. Atomic capacity guard + create as unpaid hold.
- A9. Malformed/empty JSON body → 500 (systemic, ~111 sites)** — `c.req.json()` throws `SyntaxError` before Zod on empty/non-JSON body. `check-ins.controller.ts:51` does it right `( .catch(()=>({})) )` . Add a shared `readJson(c)` → 400.
- A10. Invalid ObjectId → `CastError` 500 (systemic, ~265 sites)** — `isValidObjectId` appears nowhere. Stale/mistyped links 500 instead of 404. Shared `asObjectId(id)` guard. (`check-ins.controller.ts:15-19` is a good reference.)
- A11. Deleted-user reference crashes the "who's here now" list** — `check-ins.controller.ts:86` , `:22-24` . A checked-in user who deleted their account populates `null` ; `toPlayer(null)` throws → the venue check-in panel 500s for everyone. `rows.filter(r => r.userId)` before `.map` , null-guard `toPlayer` .
- A12. `E11000` surfaces as 500 instead of 409** — no `11000` handling anywhere. Expected "already exists" races (coach/org applications, conversation keys, registration slots) confuse users. Map `code===11000` → 409, ideally in the central error handler.

Checked & clean: external email/push/geocoding all guarded (a down Gmail/FCM does not 500 a user action); SSE cleans up intervals; messaging + roster mutations are correctly ownership-scoped; `pricing.ts` has no div-by-zero.

● **DONE — all backend findings resolved (2026-07-21)**

Plain-language summary of what was fixed in the server (API):

#	Level	What was wrong (plain language)	What we did
A1	● High	Two people booking the last court at the same instant could both succeed — an oversell.	Added a hard database rule so only one confirmed booking can exist per court + time slot; the second gets a clean "just taken" message.
A2	● High	Same double-book race for coach sessions .	Same hard rule per coach + time slot.
A3	● High	A malformed booking time could skip the price check and let someone pay any amount they chose.	Booking times are now strictly validated; a bad time is rejected before payment.
A4	● High	A crafted search query could crash the search (500) or freeze it for everyone (denial-of-service).	Search input is now treated as plain text — no crash, no freeze.
A5	● High	A coach could publish themselves into the public directory without being verified (moderation bypass).	Unverified coaches can no longer self-list; verification is required first.
A6	● High	Two players grabbing the last game slot at once could both get in (over-capacity), or a double-tap could list someone twice.	The join is now atomic — exactly one seat per slot, no duplicates.
A7	● Medium	Adding/removing roster members at the same time could lose one of the changes .	Roster add/remove are now atomic — no lost updates.
A8	● Medium	Claiming a waitlist slot created a P0 "confirmed" booking that held a court free forever , and could double-claim.	Claim is now atomic and creates an unpaid hold with a pay window that auto-expires.

#	Level	What was wrong (plain language)	What we did
A9	Medium	Sending an empty or broken request body crashed with a 500 (across ~127 endpoints).	Central fix: bad JSON now returns a clean 400 everywhere.
A10	Medium	A mistyped or stale link (bad ID) returned a 500 instead of "not found" (across ~267 endpoints).	Central fix: an invalid ID now returns a clean 400 everywhere.
A11	Medium	If a checked-in player deleted their account , the venue "who's here now" panel 500'd for everyone .	The list now skips deleted users safely.
A12	Medium	"Already exists" collisions (duplicate applications, registrations) surfaced as a confusing 500.	Central fix: these now return a clear 409 "already exists / just taken".

Verified live: new database rules built cleanly (no existing conflicts); API restarted healthy; smoke-tested — broken JSON → 400, bad ID → 404, crafted search → 200 (all previously 500). All type errors are pre-existing; the 12 fixes introduced none.

app/ — Player PWA (React 19 PWA)

Critical

P1. No React error boundary anywhere in the app `app/src/main.tsx:7-11`, `App.tsx` (`renderScreen()` switch at `:569-836`) `<StrictMode><App></StrictMode>` with zero boundaries — no `ErrorBoundary` / `getDerivedStateFromError`, no `react-error-boundary` dep. Any single screen render throw unmounts the **entire app to a permanent blank white screen**, recoverable only by manual reload. **Fix:** wrap `<AppInner>` (and ideally `<main>{renderScreen()}</main>`) in an `ErrorBoundary` with a "Something went wrong — Reload" card. **Single highest-leverage fix** — it contains every latent screen-level render bug.

High

P2. Failed owner-dashboard secondary fetches render a false "empty" state with no retry `app/src/features/owner/hooks/useOwnerDashboard.ts:85-137` `withBookings` / `withGames` / `withReviews` use `Promise.allSettled` and drop rejected results, setting no error flag (`status` tracks only the venues call). If bookings fetches fail, a busy venue shows "No bookings yet" with no retry. **Fix:** track per-list error flags; render `ErrorState` w/ retry when the request rejected.

P3. `EditClubScreen` retry dead-ends on a permanent skeleton `app/src/features/clubs/EditClubScreen.tsx:106` Retry does `setStatus('loading')` but the load effect depends only on `[clubId]` (no `reloadKey`), so the fetch never refires — the screen is stuck on the skeleton forever. **Fix:** add a `reloadKey`, bump on retry, include in effect deps (the pattern every sibling club screen already uses).

P4. Feed load failure renders a misleading "No posts yet" with no retry `app/src/features/social/FeedPanel.tsx:68-75` (empty UI `:215-219`) Initial `listFeed()` uses `.catch(() => {})` then `setLoading(false)`; a failed load silently shows the empty state on the default Social landing. **Fix:** set an `error` state in `.catch`, render `ErrorState` /retry.

P5. Match score sheet shows/submit a stale or blank scoreline across matches `app/src/features/organizer/tournaments/bracket/MatchScoreSheet.tsx:21` `BottomSheet` keeps children mounted when closed; one persistent `MatchScoreSheet` is rendered with no `key` and `rows` is `useState` initialized once, never re-synced to `match`. After entering "11-5" for match A, opening match B pre-fills B with A's "11-5"; opening a completed match shows blank. **The organizer can silently record the wrong result.** **Fix:** `useEffect(() => setRows(seedFrom(match)), [match?.id, open])`.

Medium

- P6. Leaflet markers loaded from `unpkg.com` CDN on all 6 map screens** — `FullMapScreen.tsx:11-13`, `NearbyScreen.tsx:65-67`, `CourtDetailsScreen.tsx:59-61`, `v2/NearbyScreenV2.tsx:32-34`, `owner/tabs/LocationEditorTab.tsx:22-24`, `ui/MapPinPicker.tsx:15-17`. SW caches OSM tiles but not these icons → broken pins offline/CDN-blocked. Import from bundled `leaflet/dist/images/*`.
- P7. `Chart` primitives divide by zero on sparse data** — `ui/Chart.tsx:134` (sparkline single point → NaN path), `:219-229` (donut `total===0` → NaN dasharray). Guard `points.length <= 1` and `total === 0` before dividing.
- P8. Clubs directory failure silently shows "No clubs found"** — `social/ClubsPanel.tsx:41-53` (`.catch(() => ({items:[],cursor:null}))`). Track load error, render retry.
- P9. Admin "Pricing mode" segmented control broken (undefined option ids)** — `profile/v2/SettingsScreenV2.tsx:364-375`. Passes `{value,label}` where `Segmented` expects `{id,label}`; never highlights and writes `pricingMode: undefined`. Use `id`.
- P10. Feed hide / not-interested / delete optimistic with no rollback** — `social/FeedPanel.tsx:137-143` (delete) & `:157-176` (hide/not-interested). Content stays hidden/"deleted" on failure until reload. Restore prior `posts` in `.catch`.
- P11. Chat conversation-load error is a dead-end (no retry)** — `messages/ChatScreen.tsx:399`. `<ErrorState>` with no `onRetry` (unlike `ConversationsScreen`). Add `onRetry` via a reload key.
- P12. Roster/organizer manage actions fail with zero feedback** — `organizer/tournaments/TournamentDetailScreen.tsx:64`, `organizer/openplay/SessionRosterScreen.tsx:40` (+ `ParticipantRow.tsx:27` `try/finally`), `organizer/openplay/OpenPlayScreen.tsx:87`, `organizer/venues/VenueRequestsScreen.tsx:81`, `organizer/rosters/RosterDetailScreen.tsx:66`. Approve/check-in/cancel/remove reject unhandled → button re-enables, nothing changes, no error. Add `catch` + toast.
- P13. `coachDisplay.money()` has no null guard its siblings have** — `coaches/coachDisplay.ts:13`. `amount.toLocaleString()` with no coercion; a null `b.amount / s.price` throws inside `.map` (CoachBookings/BookCoach/CoachDetail), blanking the list. `const n = Number(amount) || 0;`.

Low

- P14. `VenueOverviewTab` swallows sub-fetch failures into zeros** — `owner/tabs/VenueOverviewTab.tsx:50-72`. Failed analytics/bookings/reviews render `0/0` courts as if empty. Add a `status`, render retry when core calls reject.
- P15. `OwnerProfileScreen` has no loading/error state** — `owner/OwnerProfileScreen.tsx:87-100`. Shows "0 venues · 0 courts" on a failed load. Pull `status` / `retry` from the hook.

- **P16. `OwnerVenueScreen.reload()` swallows refresh errors** — `owner/OwnerVenueScreen.tsx:105-107` . After an edit-save, a failed refetch keeps stale data with no feedback. Toast on refresh failure.

Checked & clean: `api.ts` (`rawRequest`) wraps fetch, throws typed `ApiError` , `.json().catch(()=>null)` , checks `res.ok` , coalesces concurrent 401 refreshes, env-driven prod base URL (no hardcoded localhost); `authStore` `localStorage` parses are try/caught; polling hooks clear intervals + read fresh token (no stale closure); push/Firebase/SW init are best-effort (no boot crash); display formatters guard `NaN / Invalid Date` ; `demoState` gated behind `?demo` ; auth cold-start screens have full loading/error states; optimistic like/friend/message toggles roll back correctly.

DONE — all PWA findings resolved (2026-07-21)

Plain-language summary of what was fixed in the player mobile app:

#	Level	What was wrong (plain language)	What we did
P1	Critical	If one screen hit an error, the whole app went blank white — user had to force-reload.	Added a safety net: a single broken screen now shows a friendly "Something went wrong — Reload" card instead of killing the app. New-version errors show a "Reload to update" card.
P2	High	When the owner dashboard failed to load bookings/games/reviews, it showed "nothing here yet" as if the venue were empty.	Now it tracks each list's failure so the screen can tell "genuinely empty" from "failed to load."
P3	High	The Edit-Club screen's Retry button did nothing — stuck on the loading skeleton forever.	Retry now actually re-fetches the club.
P4	High	If the social feed failed to load, it falsely said "No posts yet" with no way to retry.	Now shows a real error with a Retry button.
P5	High	The tournament score sheet carried scores over between matches — an organizer could record the wrong result.	The score grid now resets every time you open a different match.
P6	Medium	Map pins were loaded from an outside website — broken pins when offline or if that site was blocked.	Pins now come bundled with the app; they work offline.
P8	Medium	If the clubs directory failed to load, it falsely said "No clubs found."	Now shows a real error + Retry.
P10	Medium	Hiding/deleting a feed post looked successful even when it failed — the post stayed gone until reload.	On failure the post now comes back and a toast explains it didn't save.
P11	Medium	A failed chat conversation was a dead-end (no retry).	Added a Retry button.
P12	Medium	Organizer approve/check-in/cancel/remove actions failed silently — button just re-enabled, nothing happened.	Failures are now caught and the list refreshes to show the real state.
P14	Low	The venue overview showed P0 / 0 courts when a sub-fetch failed, as if empty.	Now shows a "couldn't load X" warning so zeros aren't mistaken for real data.
P15	Low	The owner profile showed "0 venues · 0 courts" on a failed load, with no error.	Added loading + error states with a Retry.
P16	Low	After saving a venue edit, a failed refresh kept showing old data silently.	A failed refresh now surfaces an error instead of hiding it.
P7	Medium	*(reported: charts divide by zero)*	Already safe — guards already exist. No change.
P9	Medium	*(reported: pricing toggle broken)*	Already correct — control already uses the right values. No change.
P13	Medium	*(reported: coach price crash on null)*	Already safe — type safety + caller guard. No change.

Build:  passes, 0 new errors. 20 files changed. 13 fixed, 3 confirmed already-safe.

Prioritized remediation roadmap

Phase 1 — Stop the bleeding (do this week)

1. **W1 + W2** — fix the `web/` booking → checkout money path (real venue lookup; persist the booking; add a failure branch). *Nothing else matters if the core flow silently no-ops.*
2. **P1 + W3** — add error boundaries to both frontends. Converts every latent render crash from white-screen → contained, recoverable card.
3. **A3** — close the NaN price-bypass in `createBooking` (validate `startTime` / `endTime` , `Number.isFinite` guard). Security/revenue.
4. **A4** — escape search regex input (500 + ReDoS is a whole-service DoS from the search box).

Phase 2 — Systemic hardening

1. **Theme E** — shared `asObjectId()` + `readJson()` helpers; map `E11000` → 409 in the central error handler. Kills the ~265 `CastError-500` and ~111 `JSON-parse-500` classes and A12 at once.

- 2. **Theme D** — unique partial indexes / atomic updates for A1, A2, A6, A7, A8 (double-booking, game-join, roster, waitlist). Also fix waitlist's P0 confirmed hold.
- 3. **A5** — gate coach directory on isVerified (moderation bypass).
- 4. **A11** — null-guard the check-in list against deleted users.

Phase 3 — Trust & polish

- 1. **Theme B** — per-list error+retry in owner dashboard hook, feed, clubs directory (P2, P4, P8, P14, P15).
- 2. **Theme C / F** — optimistic-rollback (P10) and silent-mutation toasts (P12, W7, P16).
- 3. **P5** — tournament score-sheet re-seed (silent wrong-result recording).
- 4. Remaining Medium/Low: P3, P6, P7, P9, P11, P13, W4, W5, W6, W8, W9, W10, W11, W12.

Coverage note

Scans covered: app/ boot/infra + shared components, and feature clusters owner, social/clubs/messages/profile, organizer/tournaments/coaches/admin/home/search; api/ global middleware + all feature controllers incl. the bookings/rosters/waitlist mutation cluster; web/ in full. The app/ games / venues / bookings feature cluster (player-facing game and court-booking screens) did not return before compile — it is the one remaining area to sweep with the same rubric (null-access, missing error/empty states, optimistic rollback). The app-wide error boundary (P1) contains crash-severity there in the meantime.